XAVIER UNIVERSITY – ATENEO DE CAGAYAN

COLLEGE OF COMPUTER STUDIES
DEPARTMENT OF INFORMATION TECHNOLOGY

Project Information Monitoring and Evaluation(PRIME) System For The Alubijid Municipal Planning and Development Office

A Project Presented to the Faculty of
Department of Information Technology
College of Computer Studies

In Partial Fulfillment of the Requirement for the Course
ITCC 42 – ICT FOR DEVELOPMENT

by

Gerfel Jay Jimenez
Ivan Risan Llenares
Kent Xylle Ryz Romarate
Noel Jose Villalveto

May 15, 2026

Chapter I
Introduction

1. Background of Client

The Municipal Planning and Development Office (MPDO) of Alubijid is the main office responsible for formulating economic, social, infrastructural, environmental, institutional, and other development plans for inter-departmental coordination in pursuit of the vision and mission of the municipality. This government office is tasked with the implementation of the Annual Investment Program (AIP), the management of the projects under each program in various sectors, the assurance that the public resources of each project are fully utilized and the transparency in accordance with the government accounting standards and Commission on Audit (COA) regulations.

The Municipal Planning and Development Office (MPDO) of LGU Alubijid depends on Google Workspace and scattered spreadsheet files for monitoring, tracking of projects, and recording of data within the municipality. These files and data are saved on single computers without a central database, which makes it difficult, unreliable, and vulnerable to data loss and attacks.

Additionally, it poses a problem to have instant project status, budget utilization summary, physical progress, and appropriate visualization of data. Moreover, without a centralized database, reports can only be made after manual consolidation from multiple spreadsheets which is a time-consuming task and is subject to human error resulting in the compromise of the transparency and accountability that are necessary for the effective delivery of public services.

2. Scope and Limitation

The system's scope covers the design and development of the PRIME (Project Monitoring and Information Management System) for MPDO Alubijid. The system will feature project management, Annual Investment Program (AIP) planning, financial tracking throughout the government budget lifecycle, annual performance tracking, physical progress monitoring, issue and risk logging, GIS-based locational monitoring, document tracking integration, and an audit trail functionalities. The project will integrate with Document Tracking System (DTS) via Supabase PostgREST API.

The project excludes mobile app development, integration with all external government systems beyond document tracking services, and implementation of advanced analytics features such as predictive modeling or artificial intelligence capabilities.

Additionally, one of the major system limitations is the deliberate exclusion of the delete and edit functionalities in the user interfaces of the main data entities, a client-driven design decision to keep transparency and data integrity. Even though backend endpoints for update and delete operations have been built to allow future development and maintenance, these are not made available through the user interface, so any data entry errors of the users will become permanent and cannot be changed through the application. This limitation provides full auditability and stops data manipulation, but it demands that users be very careful during data entry and it might require administrator intervention through direct database access for corrections. The system also relies on stable internet connectivity to access the web application and external system integrations, while proper maintenance of the cloud infrastructure is necessary to ensure the systems stability.

Chapter II
Planning Phase

1. Methodology (AGILE)

The **PRIME System** was developed using the **Agile/Scrum** methodology. This framework was selected to address the evolving requirements of the MPDO, as the initial understanding of their complex financial chain and reporting workflows matured through iterative prototyping. Unlike a traditional Waterfall model, Agile allowed the team to present functional increments of the system such as the Project Registration and AIP modules to the stakeholders early on. This ensured that the final product was perfectly aligned with the transparency and accountability standards required by the LGU and COA. .

2. Project Timeline

The development followed a 12-week structured roadmap, punctuated by intensive technical sprints in mid-April 2026 to finalize the system's core infrastructure:

Weeks 1–4: Foundation and Design

Focused on requirement gathering, database schema creation in Supabase, and architectural review.

April 11, 2026: Infrastructure & Environment Setup

Work Done: Containerized the application, web, and database servers using **Docker** to create a stable, unified development environment.

Feature: System Landing Page and Initial Development environment.

April 12, 2026: Schema Finalization (Sprint 2)

Work Done: Finalized PostgreSQL models for AIP and performance monitoring.

April 13–15, 2026: Core Module Development (Sprints 3-4)

Work Done: Created the detailed Project Management view and the **Full Financial Chain** tracking system (Appropriation, Allotment, Obligation, and Disbursement).

April 16–18, 2026: API Integration (Sprint 5)

Work Done: Built and tested FastAPI endpoints for CRUD operations.

Feature: Backend services for real-time progress updates and issue logging.

April 19, 2026: UI/UX Finalization (Sprints 6-7)

Work Done: Finalized the **Figma UI Prototype v2** and integrated **Leaflet.js** for mapping.

Feature: Reporting dashboards, analytics, and GIS Map integration.

Weeks 10–12: Testing and Deployment

Conducted full system integration testing, bug fixing, and final hosting on **Netlify** and **Render**.

3. Roles & Responsibility

The project was divided into specialized roles to ensure efficiency and technical depth:

Gerfel Jay Jimenez (Database Admin & Lead): Responsible for the Supabase schema, FastAPI backend architecture, and overall milestone management.

Thirdy Villalveto (Frontend Developer): Managed React component development for the Landing Page and System Management, as well as feature branches for documents and alerts.

Ivan Risan Llenares (Reporting & Analytics Developer): Specifically tasked with the **Issue & Risk Log**, **Audit Trail**, risk analytics, and the generation of downloadable A4 reports.

France, Luke, Kent, Brod, and Rey (Reporting/Analytics & Feature Leads): Handled specific modules including Budget Utilization, Gantt Charts, Monthly Reports, and Map Monitoring.

All Members: Collaborated on UI/UX design, dashboard layouts, and technical documentation to ensure system-wide consistency.

Chapter III

Analysis Phase

1. Data-Flow-Diagram(DFD)

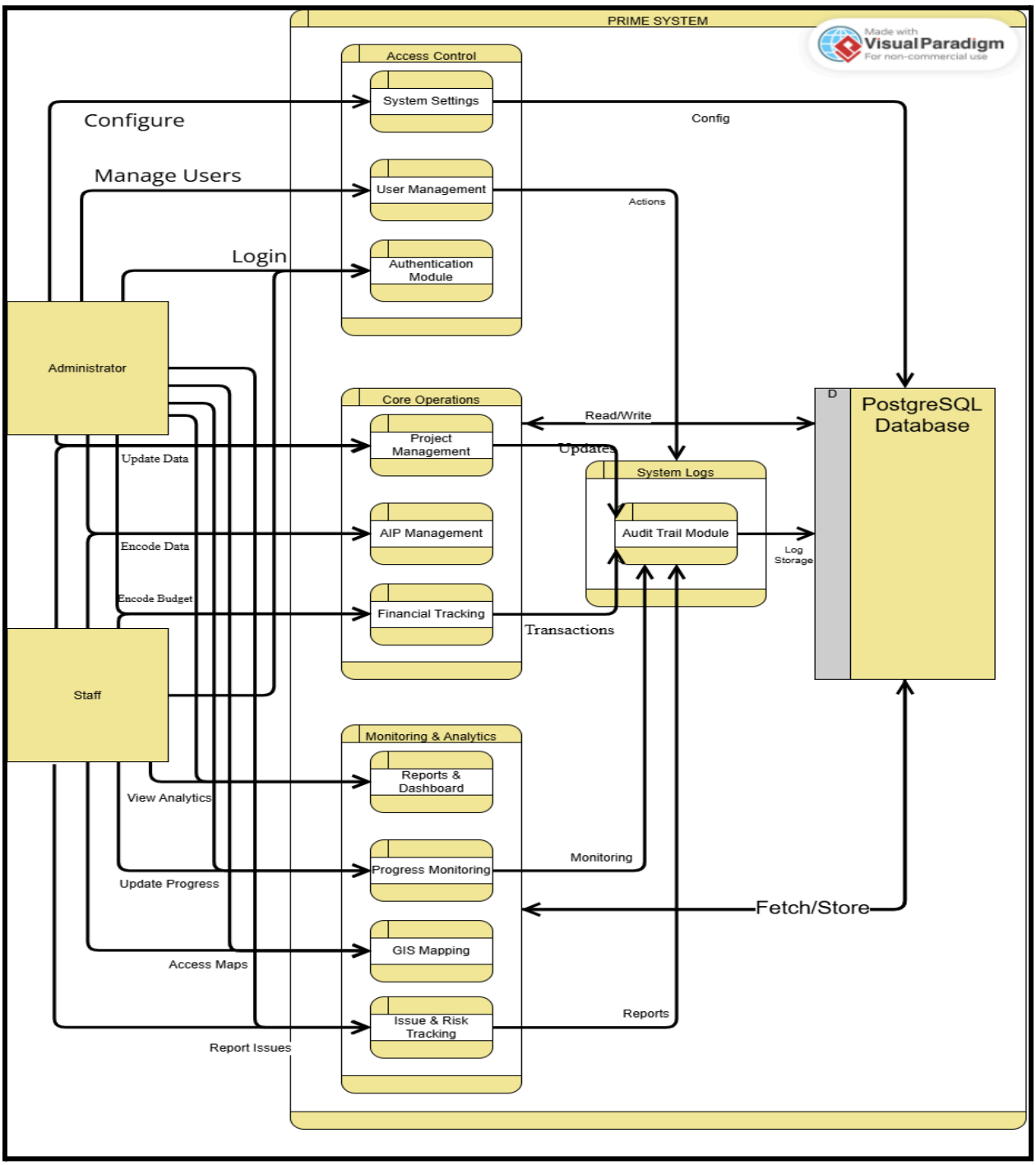


Figure 1: Overview of the PRIME System data flow and database interaction

2. Use case

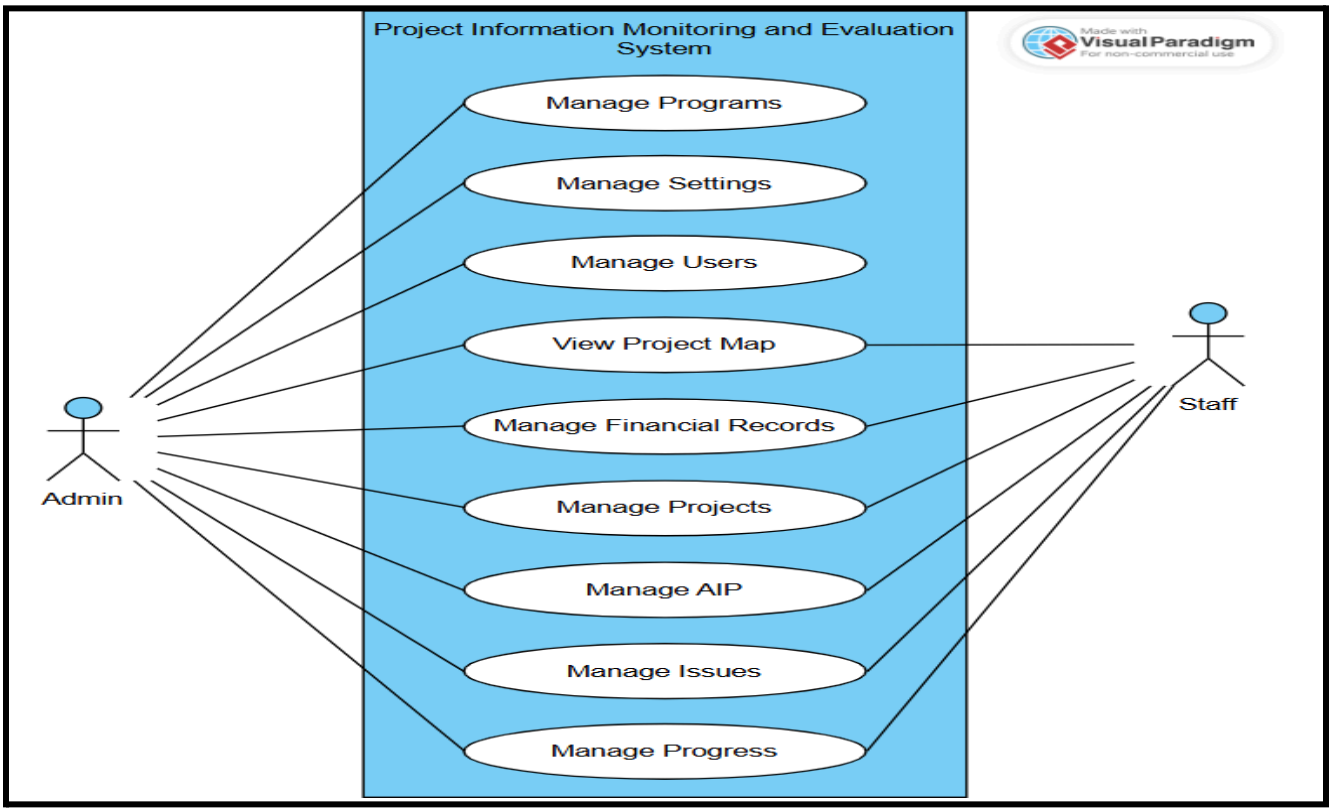


Figure 2: General use case overview of PRIME System functionalities

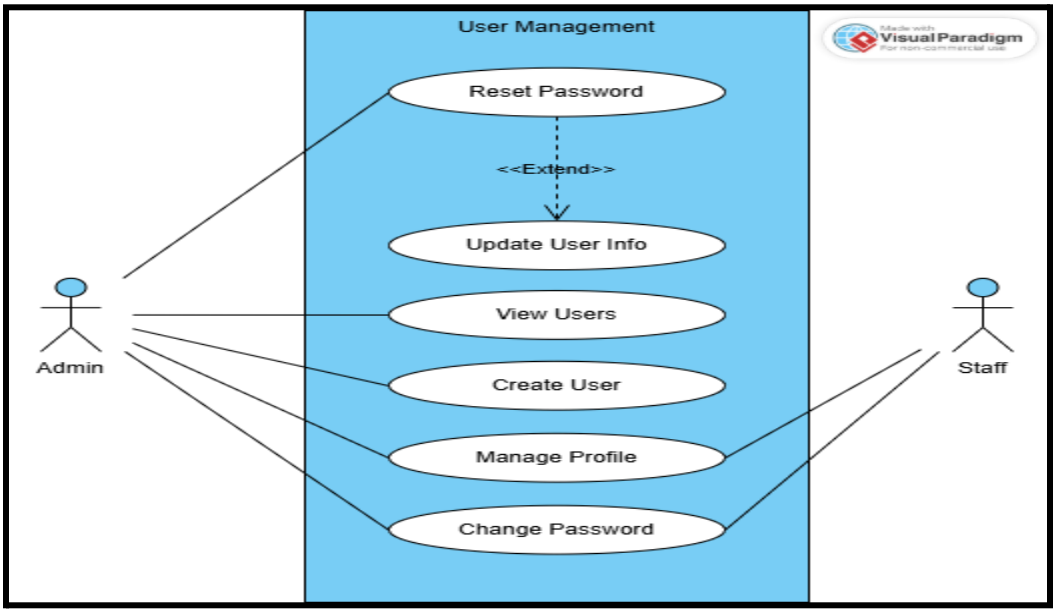


Figure 3: User account and profile management use cases

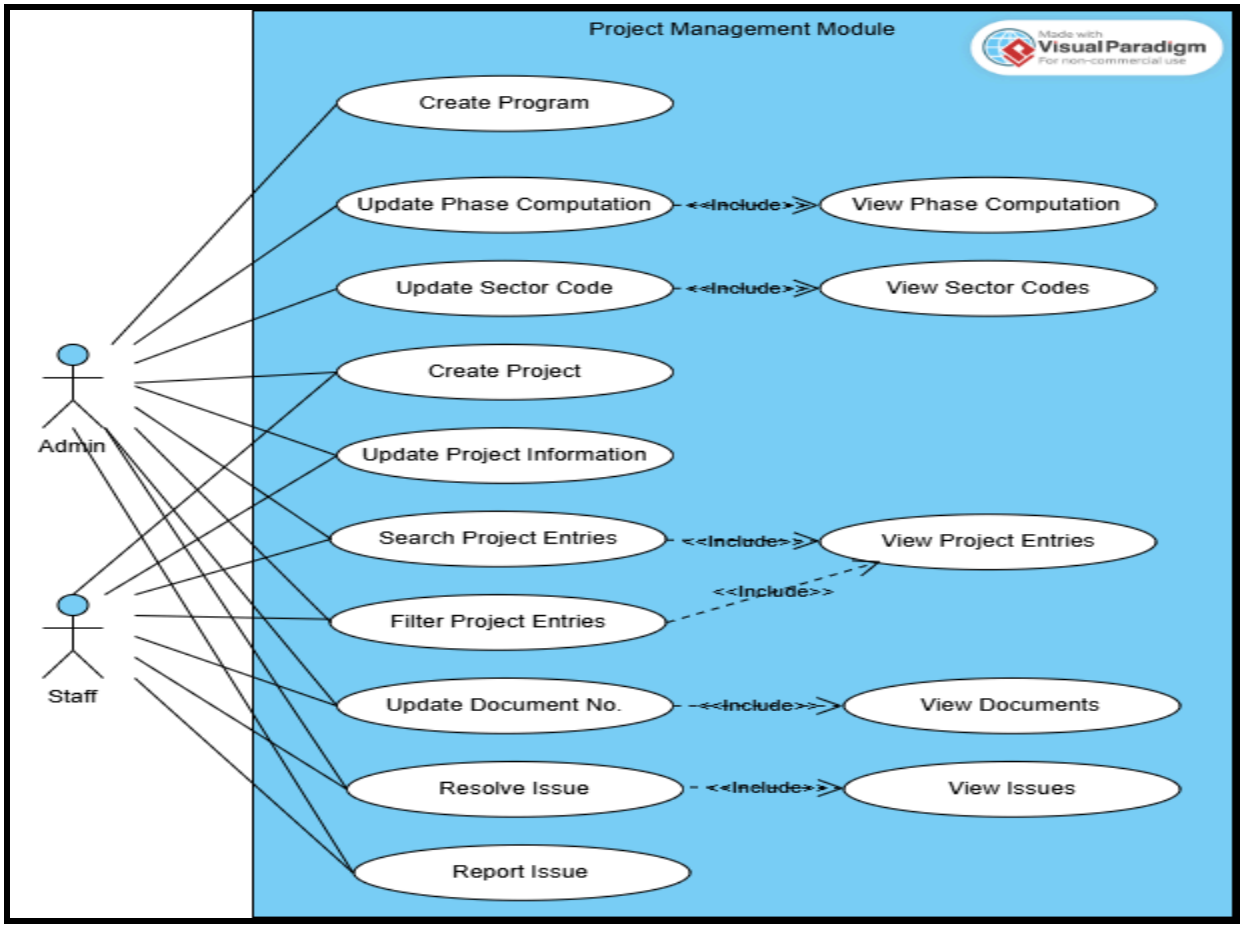


Figure 4: Project management process and monitoring use cases

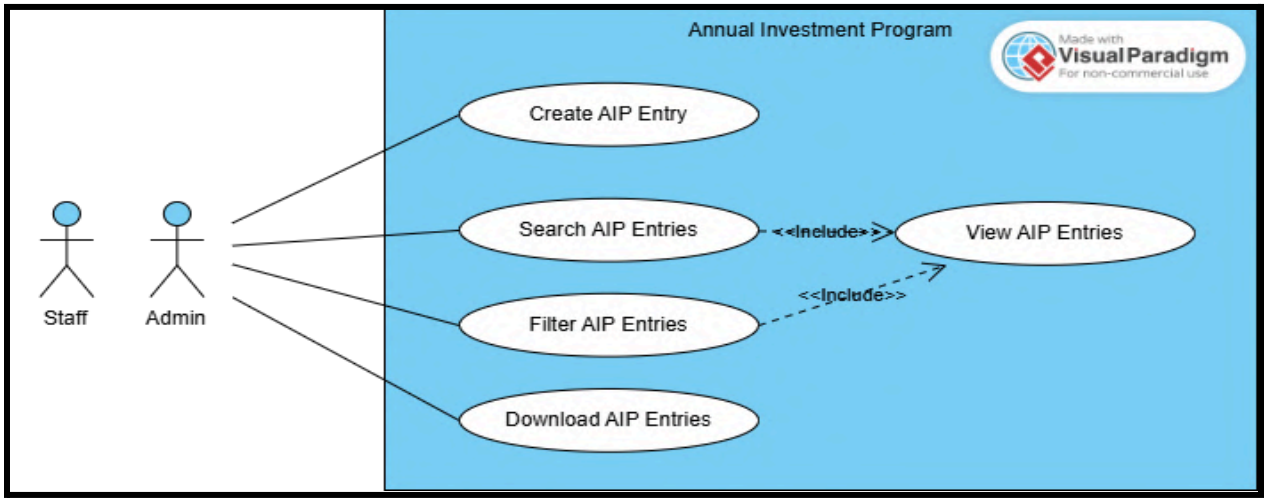


Figure 5: Annual Investment Program (AIP) management use cases

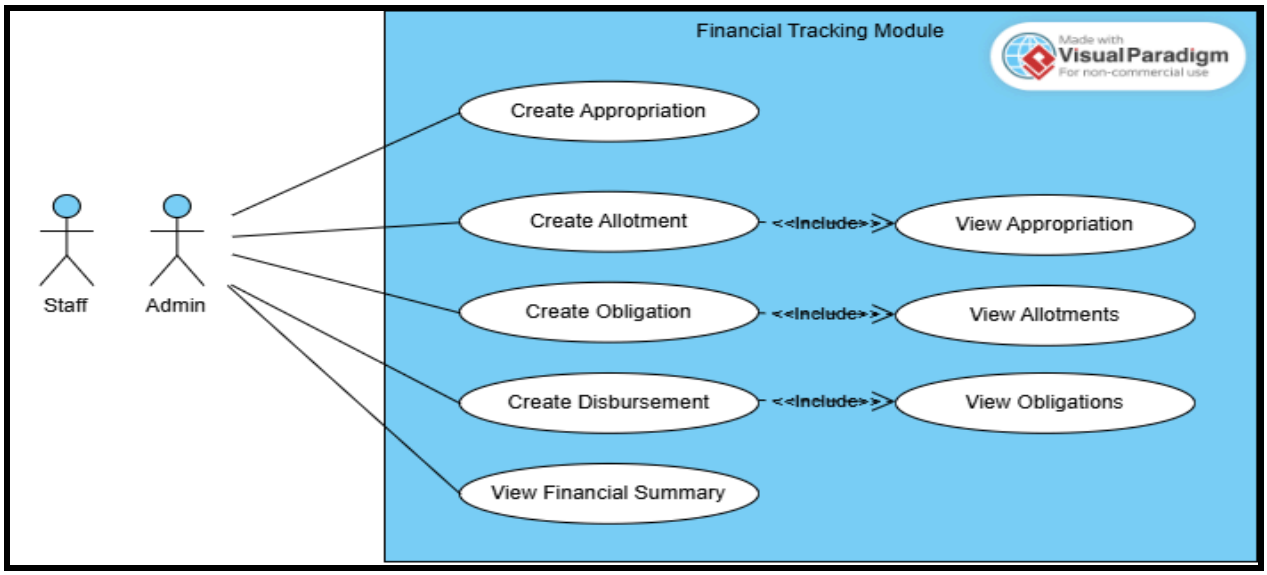


Figure 6: Financial tracking and budget monitoring use cases

3. Infrastructure Setup

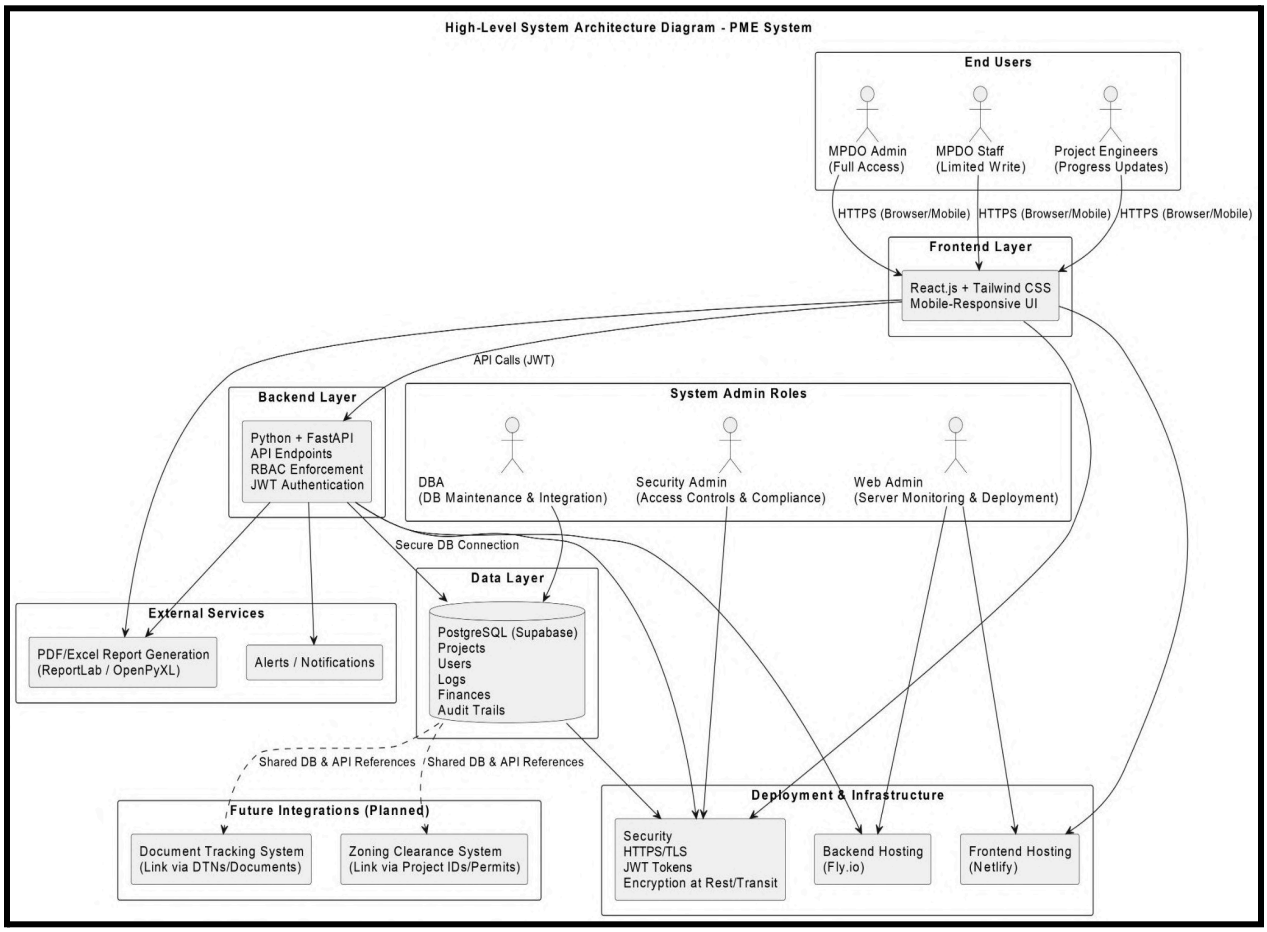


Figure 7: Overall infrastructure architecture of the MPDO PRIME System.

4. Functional and User Requirements

The PRIME System is intended to provide a consolidated digital platform in place of manual spreadsheet-based procedures.

Functional Requirements

These requirements define the specific behaviors, services, and functions that the system must provide to allow MPDO Alubijid to manage its operations effectively.

- **AIP Management:** The system must produce structured Annual Investment Program (AIP) entries with automatic reference coding and classification based on municipal sectors.

- **Financial Tracking:** The system is expected to keep track of the whole budget lifecycle including appropriation, allocation, obligation, and disbursement while making sure that government accounting standards are adhered to.

- **GIS Integration:** The system must provide visible map markers that will facilitate real-time locational monitoring and will allow the users to geographically track municipal projects.

- **Physical Progress Tracking:** The system must be able to compute and portray project completion percentages and also produce visual Gantt charts based on fiscal year performance indicators.

- **Audit Trail:** As deletion and editing are not allowed in the main user interface to safeguard transparency, the system is required to log every data creation and update (including user identity, action performed, and timestamp).

- **Issue and Risk Logging:** The system should permit the users to chronicle project bottlenecks such as weather delays or material shortages as well as to record the corresponding corrective measures.

- **Document Tracking Integration:** The system needs to be integrated with the current Document Tracking System (DTS) through API so that the digital project records could be linked to their physical document trails.

User Requirements

These requirements highlight the end-users (MPDO Admin and Staff) needs and also set a limit on how much the system can change to keep it both accessible and secure.

- **Accessibility:** The system should be accessible to users via any major web browser as long as the internet connection is reliable. This is to prevent users from depending on software installation on their machines.

- **Security & Authentication:** The system must implement a Role-Based Access Control (RBAC) and use JSON Web Token (JWT) for authentication so that only authorized individuals have the ability to access or enter sensitive municipal data.

- **Data Integrity:** The system must have a means to automatically back up data (GitHub Actions) in the cloud so as not to encounter data losses that are typical when storing data in spreadsheets on single computers.

- **User-Friendly Interface:** In order to make the dashboard and project tables conveniently accessible on different devices, the system should be designed responsively and use modern UI components (shadcn/ui).

Chapter IV

Design and Development Phase

1. Design Plan

The MPDO PRIME System follows a modern three-tier web architecture with a clear separation of concerns between frontend, backend, and database layers. The system is designed as a unified project monitoring platform for the Municipal Planning and Development Office (MPDO) enabling them to track municipal projects, budget, progress, and documentation without any delay.

The design emphasizes:

- Role-based access control for employees and administrators
- Immediate data synchronization across project phases, financial tracking, and progress monitoring
- Geospatial features for project location tracking
- Simple audit logging for responsibility
- The user-friendly web interface can be used on all types of devices

2. System Specification

Frontend Layer

- Built with React 19 and TypeScript for type-safe, maintainable code
- Uses Vite as the build tool for fast development and optimized production builds Styled with TailwindCSS for a consistent, responsive design system
- Implements TanStack Router for client-side navigation and TanStack Query for efficient data fetching
- Features interactive data visualization using Recharts and Leaflet.js for map-based project tracking.

Backend Layer

- Powered by FastAPI, a Python web framework
- Uses SQLAlchemy ORM for database operations with PostgreSQL 15
- Implements JWT-based stateless authentication with bcrypt password hashing
- Features rate limiting and CORS middleware for security
- Auto-generates interactive API documentation via Swagger UI at /docs.

Database Layer: PostgreSQL 15 relational database hosted on Supabase

3. System Layout

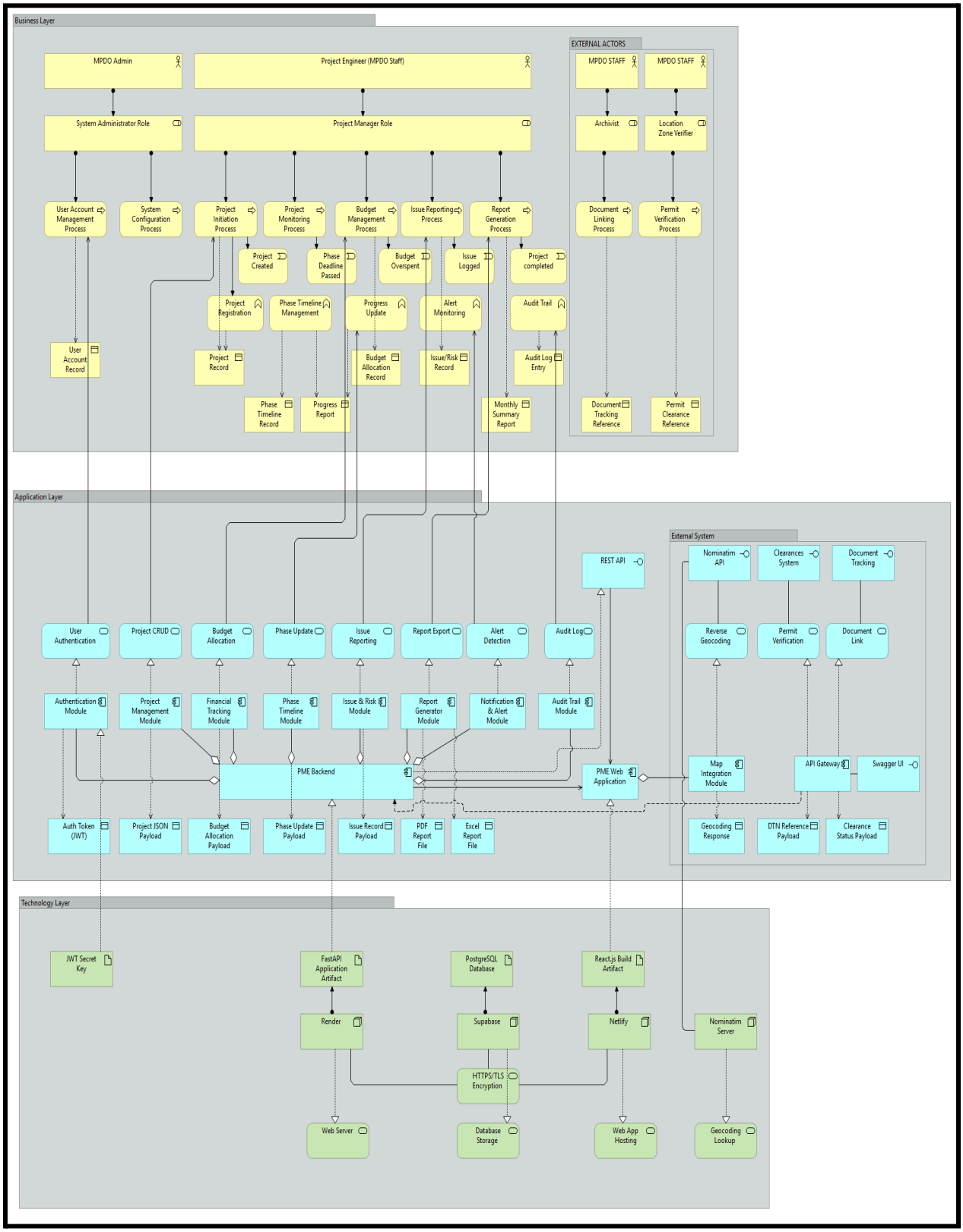


Figure 8: Solution Architecture of Project Information Management and Evaluation System

Chapter V

Testing and Evaluation Phase

1. User Acceptance Testing (UAT) - Test Plan

   Test Case Spreadsheet Link *MPDO PRIME SYSTEM TEST CASES*
   The one failed test cases were due to incomplete system integration for the Locational Clearance System. These are configuration-related issues and not caused by defects in the application itself.

   Passed – The system is considered ready for deployment.

2. Feedback from the Test

   Overall, the testing session was a huge success, and the users had a lot of great things to say about how amazing and well-thought-out the system is, especially the aip reference coding structure. However, there were a few spots where they got a bit mixed up and suggested to be fixed. Below is the list to be fixed.

   - To fix reloading of project that made the netlify web app push an error
   - To remove actual start and end date fields in edit project modal, and make it automatic based on phase progress.
   - To use the fiscal year physical progress/performance as the computation of the Gantt Chart tab.
   - To change the date filters of MM-DDDD-YYYY which they have asked for, to use only the month and year.
   - Make the user's full name to be used in financial audit logs.
   - Lastly, to allow adding of users which have an error during the test.

3. Future Recommendation for Improvement
   - Improvement of the logic used for updating project lifecycle status.
   - Enhancement of budget financials visualizations through advance line graphs to better represent the expenses done within the defined period of time.
   - Configuration of map markers to support visibility toggling, improving user experience when markers are positioned in close proximity.
   - Alerts for delayed projects and unused budgets

Chapter VI

Maintenance and Support Phase

1. Manual of Policies & Procedure

Herein provided below are the operational policies and procedures that shall be the basis for the administration as well as the day-to-day usage of the PRIME System. All personnel who have been given rights to access the system are supposed to abide by these rules to maintain the integrity of the data, security, and accountability.

A. User Access Management

MPDO Admin is the sole manager of the user accounts in the PRIME System. For setting up a new account, the following information must be gathered: full name, username, email address, password, assigned role, and office assignment. For disabling accounts, a soft-deletion strategy is employed — the account is flagged as inactive instead of being deleted permanently. This way, the audit trail and historical records linked to the user are preserved.

Changing of forgotten passwords is done by the Administrator only. An Admin user can reset another user's password via the system's account management interface. A user can also change their own password via the profile settings.

B. Data Entry Standards

The system deliberately excludes delete and edit functionalities on main data entities to preserve transparency and auditability. So, all users must be very careful while entering data since mistakes cannot be fixed directly on the web page. Fixes of data entry errors will require database admins intervention through direct database access. Such cases must be documented and approved before proceeding.

Project Creation and Update Procedures
Projects entered into the PRIME System should be actual ones being proposed by each office of LGU Alubijid. Project records have a government-standard ID format automatically generated when created. All staff must check their entries before submission, as the system prohibits deletion or direct editing the classification of projects once it is submitted.

The following information must be accurately filled in during project creation:

- Project title and description
- Target location with coordinates for GIS mapping
- Assigned sector, program, and implementing office

Changes to project data - such as phase progress and issue reports - should only be submitted after verifying the facts on the ground. For entries containing errors, Admin has to be informed for making corrections via direct database access, as the UI does not support editing of submitted records.

Budget Classification Guidelines

Money-related data recorded in the PRIME System must be aligned with the following budget classifications of the Philippine government:

- **Personal Services (PS)** - personnel remuneration and other employee benefits
- **Maintenance and Other Operating Expenses (MOOE)** - operational day-to-day costs like supplies, utilities, and services from contractors
- **Financial Expenses** - bank fees, interests, and other costs of financial transactions
- **Capital Outlay (CO)** - purchasing assets, building infrastructure, and equipment

Monetary records should correspond to the government budget lifecycle where arrangement is necessary: First, **appropriation** is created then the **allotment** comes later, and a recording of an **obligation** is only possible if there is already an allotment. **Disbursement** entries are made only after obligations are confirmed.

Performance Indicator and Target Documentation Requirements

The AIP entry for each project must specify the physical target to be completed in that fiscal year. These entries assist the system in calculating progress percentages seen on Gantt charts. Staff should ensure that the expected targets are attainable, verifiable, and adhere to the agreed-upon project scope. After total targets have been specified, they can still be changed using the UI on the Physical Progress Card; however, an actual progress (performance report) must be provided quarterly for the AIP entering the fiscal year.

Issue Logging and Resolution Tracking Protocols

Should a project face delay, risk, or a problem during implementation, it must be logged without delay in the Issue and Risk Tracking module. Each issue entry has to be accompanied by:

- categories of issue identified such as weather delay, material shortage, contractor-related, etc.
- a short description of the issue
- record of when the problem was identified
- any corrective action that has been taken or resorted to

C. System Usage Policies

Login and Session Management Procedures

Users are required to access the system using their registered email and password only. Sharing one's credentials with other people is forbidden since every activity done within the system is documented through an audit trail and is associated with the user account that did it.

Data Backup and Recovery Responsibilities

Backups of the database are managed automatically through the system's GitHub Actions workflow, which is scheduled to run every Monday to Thursday at 6:00 PM. The Admin's role includes occasionally checking that backups are being made successfully by viewing the workflow run status on the repository. When data is lost, the recovery operation has to be carried out by either the database administrator who have been given the right to use the last available backup located in the repository.

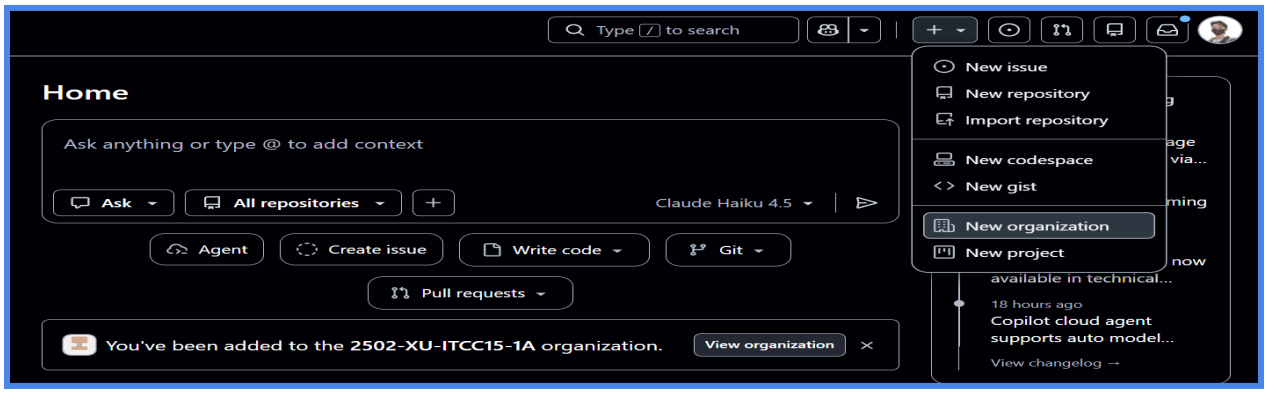
Audit Log Monitoring and Review Schedules

The Audit Trail Page keeps track of all operations that lead to data creation or update along with the user who performed it and the timestamp. The Admin must review the audit logs monthly to identify any unusual or unauthorized entries. All inconsistencies that are discovered should be communicated and must be taken up through the proper line of authority, within LGU.

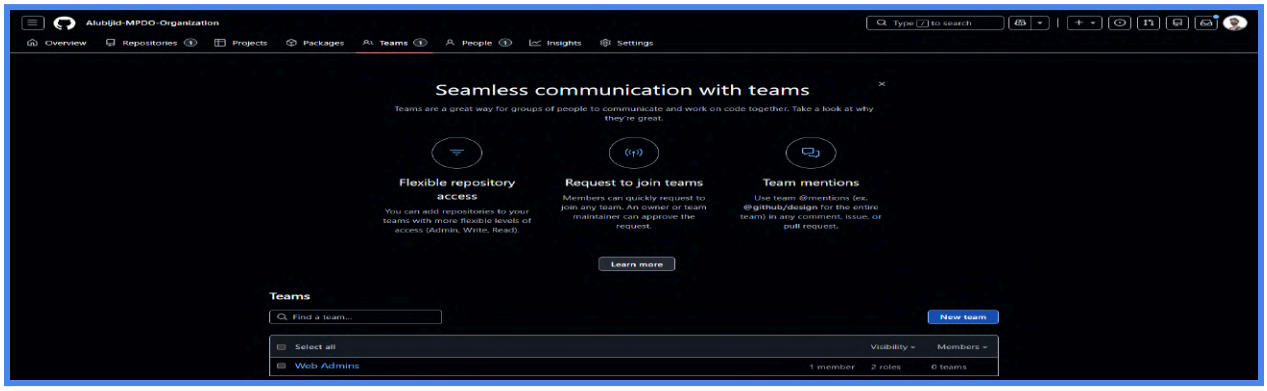
D. Deployment Procedures

1) Set up the GitHub organization

1. Create or log in to your GitHub account.
2. In the top-right corner of GitHub, click the **+** icon and create a new **Organization**.



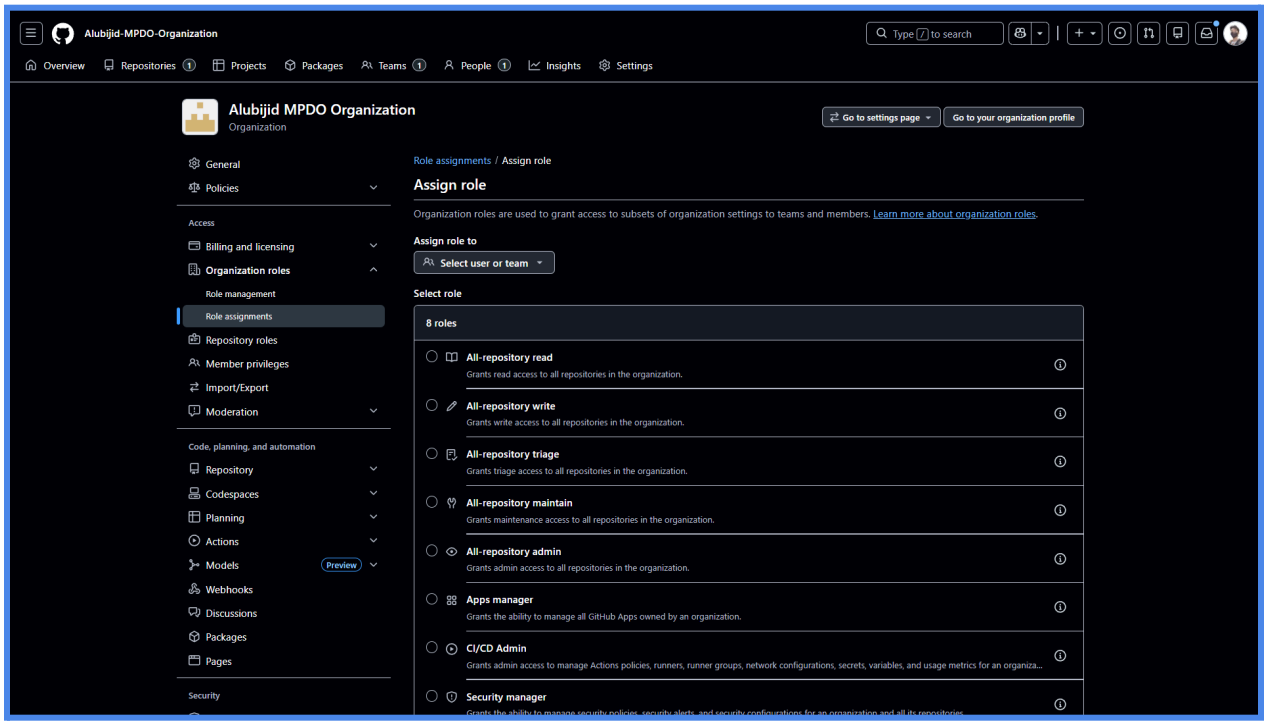
3. Choose the free plan. And enter the organization name, contact email, and the official organization name.
4. Open the organization, go to **Teams**, and create these teams:
    - **Web Administrators**
    - **Database Administrators**
    - **Security Administrators**
    - **Developers**



5. Add the team members who will handle each part of the deployment.

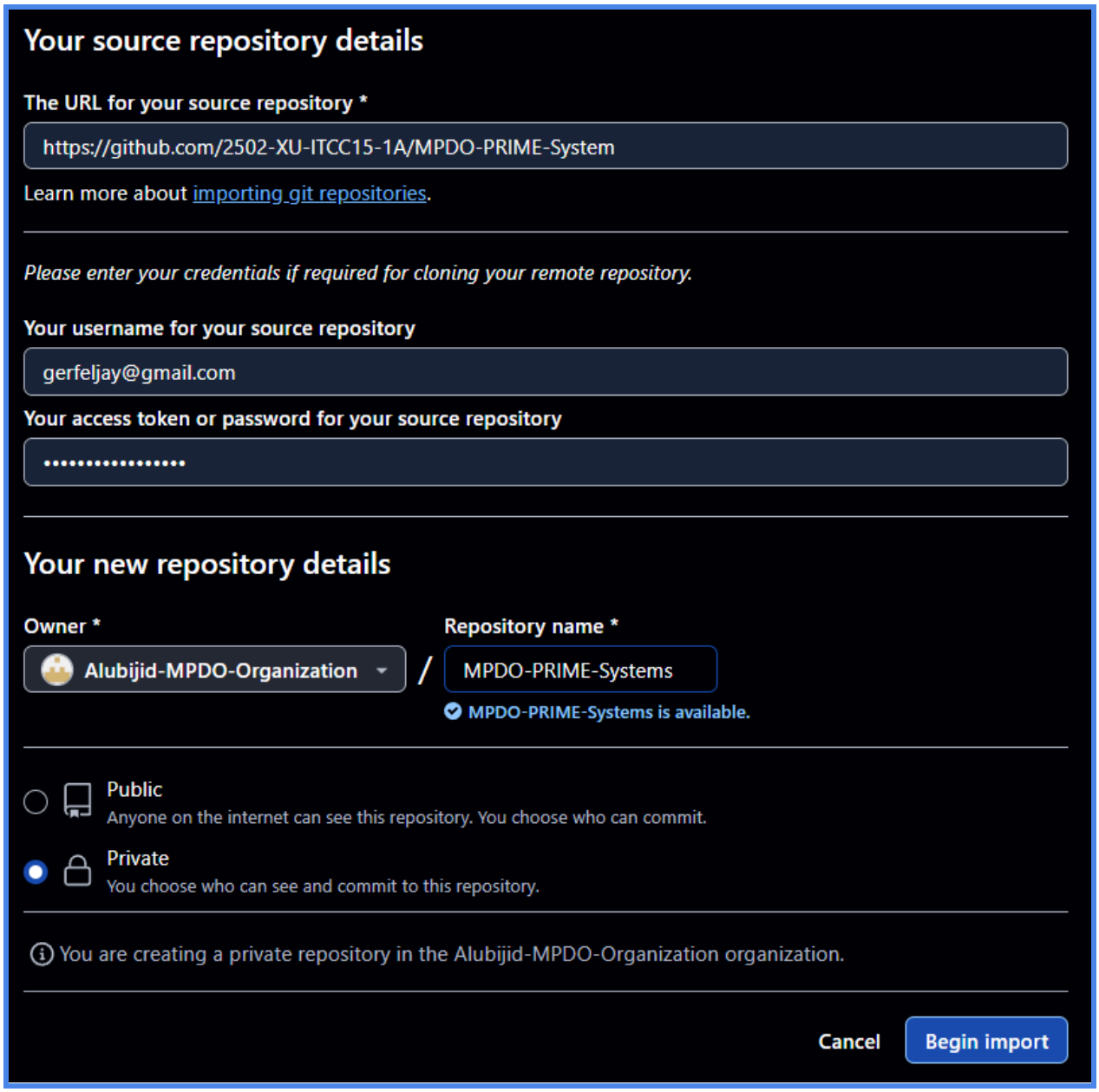
2) Assign roles

1. Go to the organization **Settings**.
2. Open **Organization roles** and then **Role assignments**.
3. Assign these roles:
    - **Web Administrators → All-repository maintain**
    - **Web Administrators → CI/CD admin**
    - **Database Administrators → All-repository write**
    - **Security Administrators → Security manager**
    - **Developers → All-repository write**



3) Import the codebase

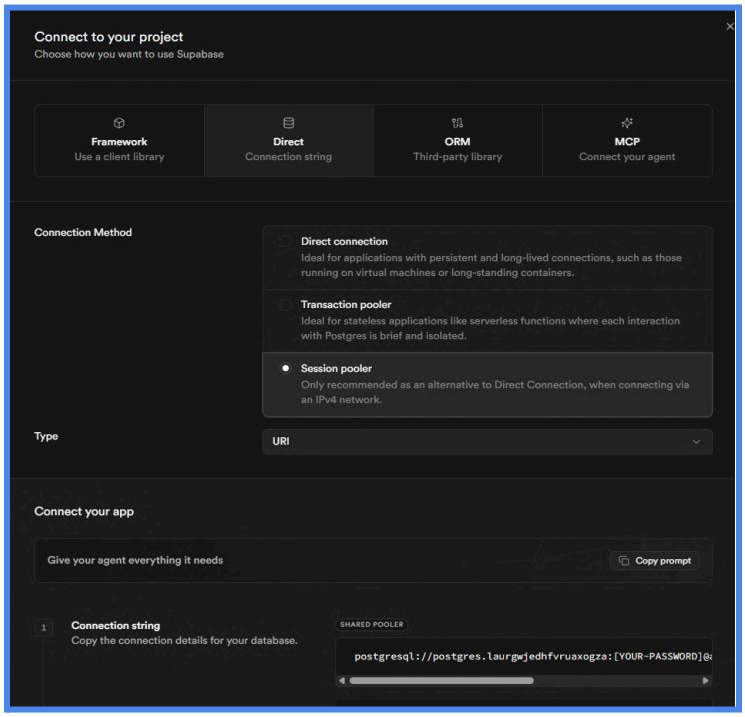
1. In GitHub, click the **+** icon again and choose **Import repository**.
2. Use this source URL:
   https://github.com/2502-XU-ITCC15-1A/MPDO-PRIME-System
3. Skip the username and access token fields.
4. Select your organization as the owner.
5. Set the repository name to **MPDO PRIME System** or your preferred name.



6. Start the import and wait until it finishes.

4) Deploy the database in Supabase

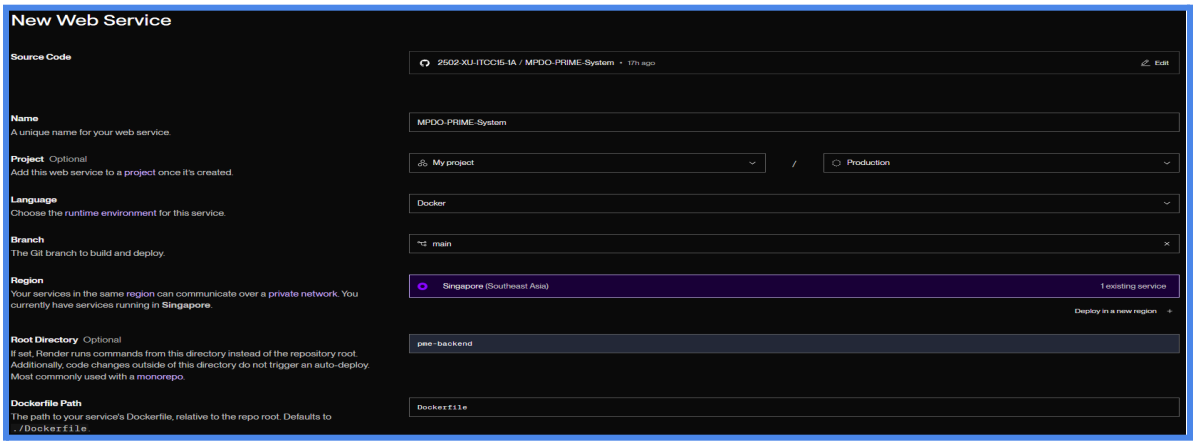
1. Create or log in to your **Supabase** account using GitHub.
2. Create a new organization if needed.
3. Create a new project.
4. Authorize GitHub and select the imported repository.
5. Set your preferred project name.
6. Create a database password and keep it stored, because you will need it later.
7. Keep the region in **Asia Pacific**.
8. Enable automatic RLS so the database is not publicly exposed.
9. Create the project.
10. Open the project, go to **Connect**, then choose the **direct connection string** and **session pooler**.



11. Copy the connection string and insert your password into it. This becomes your DATABASE_URL.

5) Deploy the backend in Render

1. Sign in to **Render** using GitHub.
2. From the dashboard, create a new project.
3. Set the project name to **MPDO PRIME** or any preferred name.
4. Click **+ New** and choose **Web Service**.
5. Connect your GitHub account and select the imported repository.
6. Set the deployment details:
    - **Language**: Docker
    - **Branch**: main
    - **Region**: Singapore
    - **Root Directory**: pme-backend
    - **Dockerfile Path**: Dockerfile



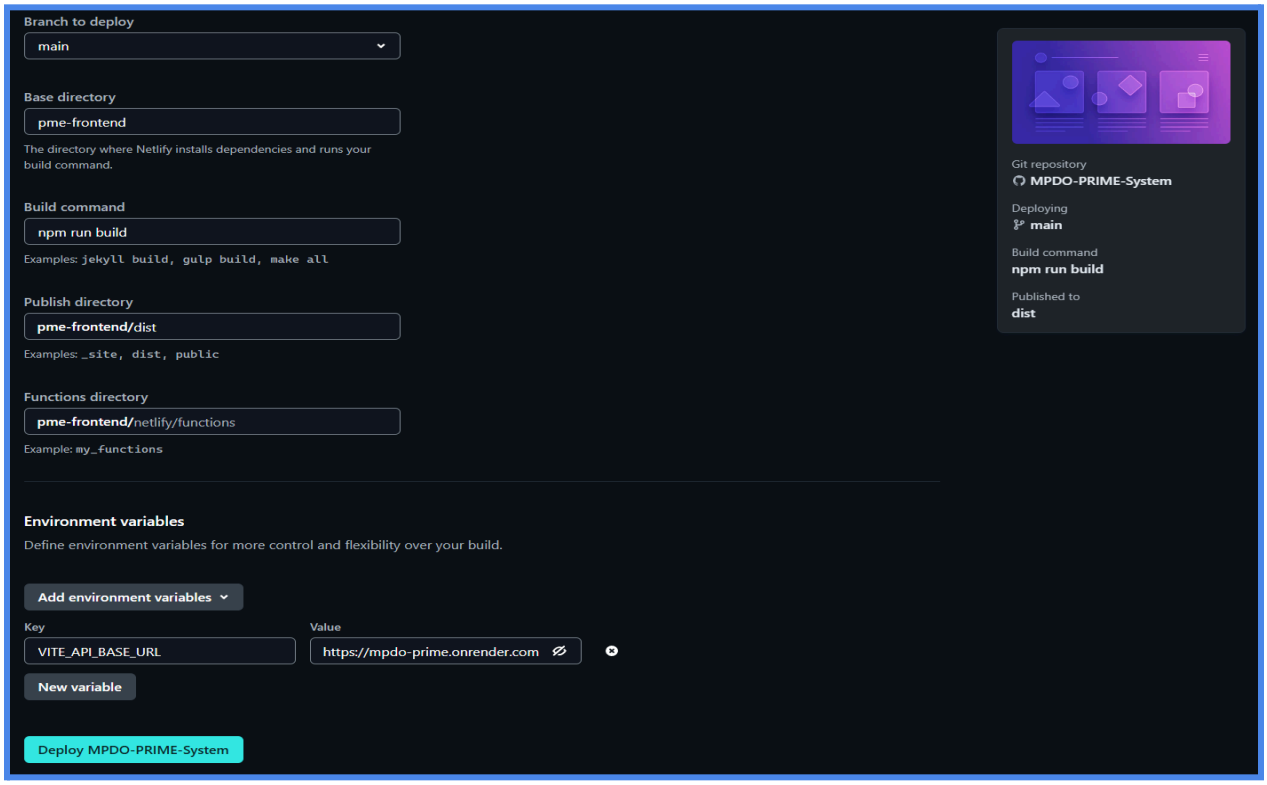
7. Add these environment variables:

```
DATABASE_URL=supabase connection string
JWT_SECRET_KEY=xxx
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60
REFRESH_TOKEN_EXPIRE_DAYS=7
FRONTEND_PME_URL=netlify frontend url
DOCUMENT_TRACKING_API_BASE_URL=dts supabase
postgrest api url
DOCUMENT_TRACKING_API_KEY=dts supabase secret key
EXTERNAL_API_TIMEOUT_SECONDS=10
```

8. Deploy the web service.

6) Deploy the frontend in Netlify

1. Sign in or log in to **Netlify** using GitHub.
2. In the main dashboard, click **Add new project**.
3. Choose **Import from GitHub** and authorize Netlify to access your GitHub account.
4. Select your organization and the imported repository.
5. Set the project name, which will also be used as the default URL.
6. Set the build details:
    - **Branch to deploy**: main
    - **Base directory**: pme-frontend
    - **Build command**: npm run build
    - **Publish directory**: pme-frontend/dist



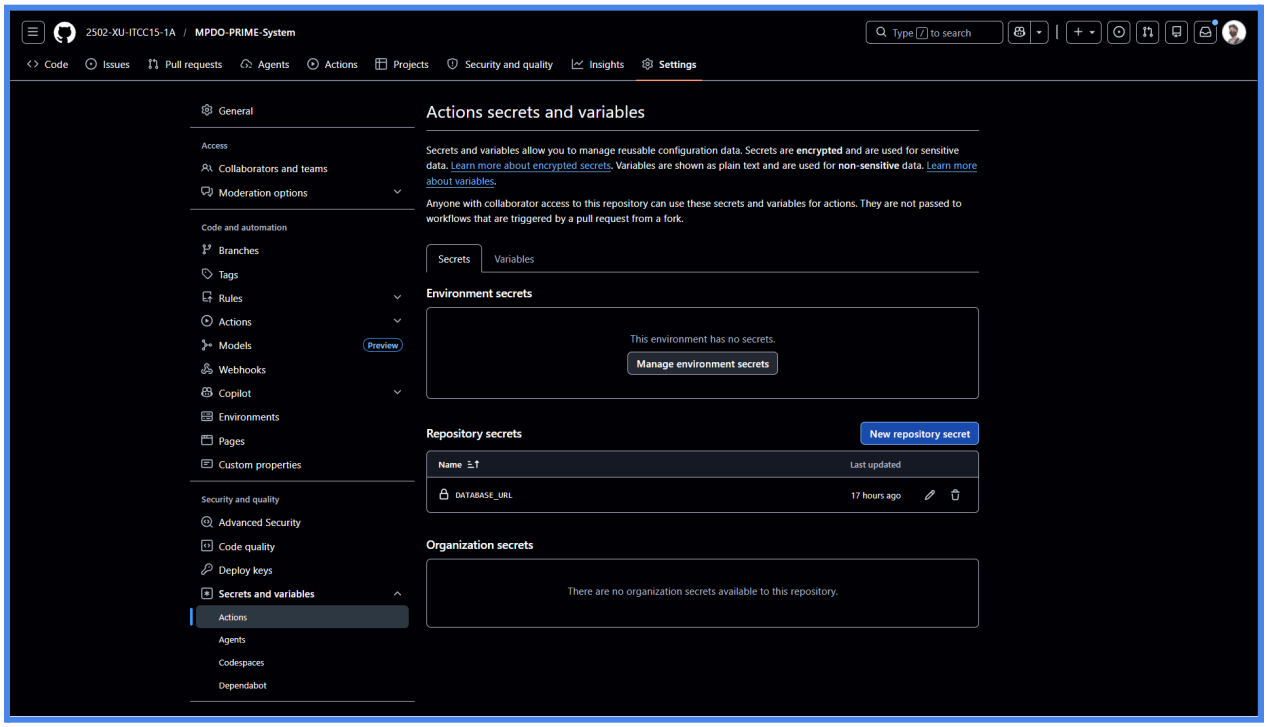
7. Add this environment variable:
    - VITE_API_BASE_URL = your Render web service URL
8. Deploy the site.

7) Final update

1. Copy the new Netlify URL.
2. Go back to your Render web service.
3. Update FRONTEND_PME_URL with the Netlify application URL.
4. Redeploy the backend so the new value takes effect.

To enable daily backup

5. Go back to Github imported repository
6. Go to Repository Settings Tab.
7. Scroll down to the Secrets and variables under Security and quality.
8. Add a New repository secret, and name it DATABASE_URL with the session connection string from supabase.



2. Maintenance Plan

a. System Overview

User Roles and Permissions

MPDO Admin

The MPDO Admin role is granted full access in the PRIME System, enabling complete oversight of all municipal development projects. This includes creating and managing projects, handling user accounts, configuring system settings, viewing all data and reports, and ensuring full accountability across the LGU Alubijid's planning and development activities.

MPDO Staff

The MPDO Staff role operates with limited write access in the PRIME System, allowing project managers to create and manage assigned projects, view all data and reports, update project progress, and tracking of project financials.

System Capability

- Project creation with sector-based classification, programs, and office
- Real monitoring through the use of visual status categorization and thorough project tracking capabilities across all municipal projects.
- Auto-ID creation that follows the structured format of the government
- Financial tracking capabilities such as budget appropriation, allotment releases, obligations, and actual disbursements
- Phase and Physical progress tracking by using completion percentages
- Recording of issues and risks such as bottlenecks, weather delays, or material shortages along with its corrective actions
- Document tracking features with manual-linkaged to official tracking IDs and physical document trails
- GIS-based location monitoring to ensure that projects are implemented at the intended locations
- Annual Investment Program (AIP) management along with performance indicators and target tracking.

Technologies and Platforms

The system's user interface is built using **Frontend React.js**, which provides a clean and mobile-responsive layout. This is achieved with the help of **Tailwind CSS**, making it accessible for both office staff using desktops and field engineers using mobile devices.

On the other hand, **Backend Python** is used along with the **FastAPI** framework to handle all the server-side logic. The choice of FastAPI is due to its high performance, ability to automatically generate API documentation, and ease of defining secure endpoints. Once the system is deployed, this documentation will be accessible at a specified URL, with all endpoints having a base URL for easy reference.

For data storage, **PostgreSQL Supabase** is utilized as the centralized relational database. It stores a wide range of data, including project records, user accounts, progress logs, financial data, and integration references. Essentially, it serves as the single source of truth for all connected municipal systems.

In terms of deployment, the system will be hosted on two platforms: **Render** for the backend API and **Netlify** for the frontend. These platforms offer 24/7 availability, easy deployment pipelines, and scalable infrastructure, making them suitable for a government system that is expected to grow over time.

Security is a top priority, with authentication handled via **JSON Web Tokens** (JWT). All API endpoints are protected, requiring a valid token for access. Additionally, the Security Administrator is responsible for configuring and enforcing encryption (HTTPS/TLS) for all data in transit and ensuring that data stored in the database is protected at rest.

PRIME Infrastructure Diagram Overview

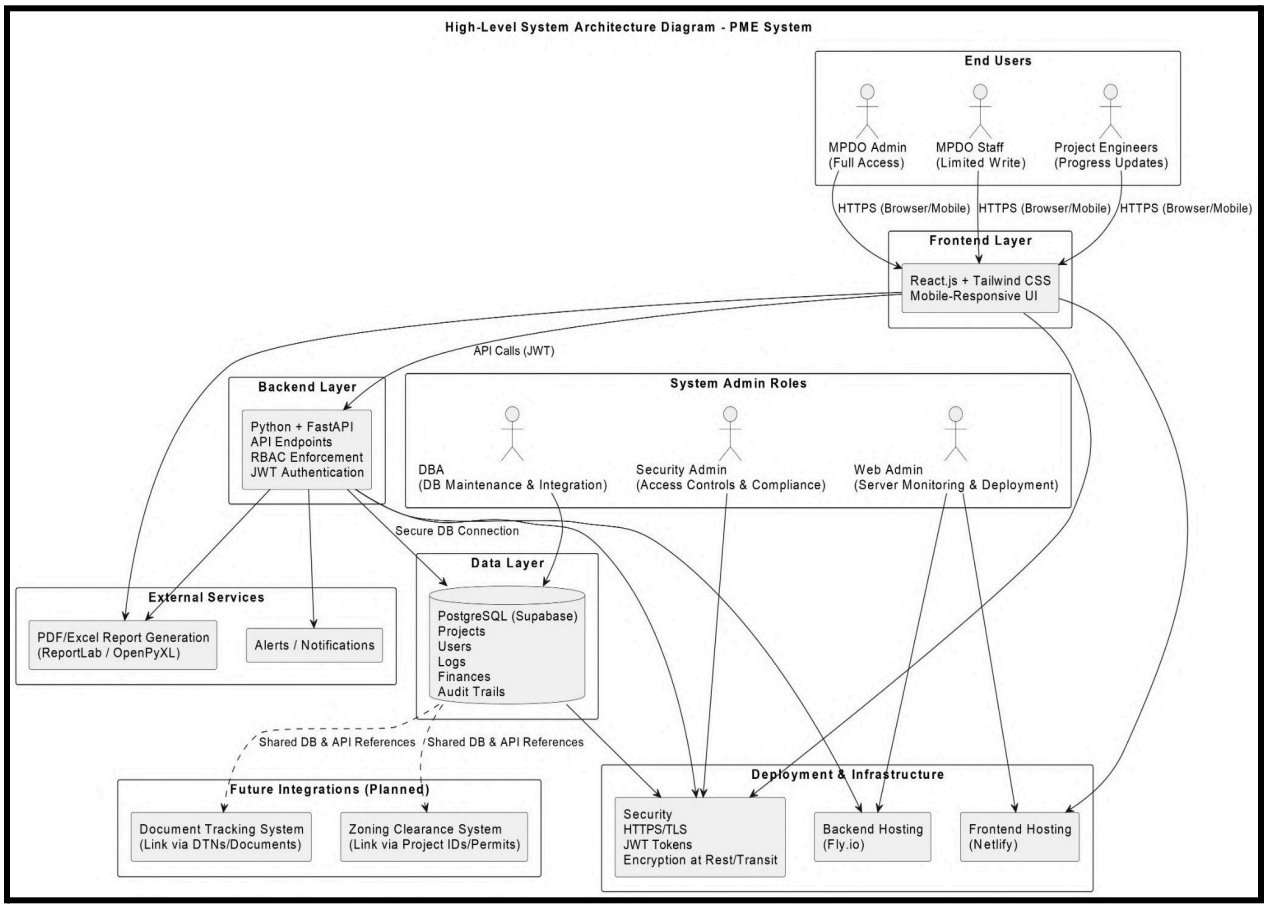


Figure 9: Infrastructure Diagram Overview of MPDO PRIME System

b. Maintenance Types and Tasks

Preventive Maintenance

API Health Monitoring — Backend API endpoints must be regularly tested to confirm that the services are running efficiently and the responses given are accurate. The APIs of the main endpoints such as project monitoring, financial tracking, and analytics modules should be tested at least once a week.

Daily Database Backup — The backup system uses a GitHub Actions workflow named backup.yml that is automated and runs Monday to Thursday at 6:00 PM UTC. The workflow compresses a backup copy of the PostgreSQL database hosted on Supabase. The backup files are stored as GitHub Actions artifacts for seven days. The system administrator must regularly check that the backups have been successful.

Dependency Updates — The frontend and backend dependencies have to be frequently checked for new updates especially for security and bug fixes. This includes the main frontend and backend libraries: React and Vite for frontend and FastAPI and SQLAlchemy for backend.

Corrective Maintenance

This kind of maintenance is done when errors, bugs or system problems are detected after deployment.

Database Schema Migration — When a new feature necessitates a change in the database structure, Alembic migrations must be made and executed correctly. Before deployment to production, the migration scripts must first be tested in the local development environment.

Bug Fixes — All bugs and errors in the system that are reported must be documented, tested, fixed and redeployed following the regular deployment process.

Data Corrections — Since UI editing and deletion are not allowed, the administrator will have to do any necessary corrections in the database directly with adequate documentation.

Adaptive Maintenance

Adaptive maintenance is carried out when external systems, services, or environments are changed and the system has to be correspondingly changed.

DTS Integration Updates — If the changes in the Document Tracking System API involve database URLs, API keys, or schema structure, the environment variables and the integration files have to be updated as well.

Supabase Configuration Changes — If database connection settings or Supabase project configurations are changed, the related environment variables in GitHub Actions and Render must be updated without delay.

Platform Changes — Changes in platforms such as Netlify, Render, or Supabase may result in deployment settings, build commands, or environment configurations having to be updated.

Perfective Maintenance

Perfective maintenance focuses on improving system performance, usability, and user experience based on feedback and future requirements.

Search and Filter Improvements — Project and financial pages may get extra search and filtering capabilities to facilitate record retrieval.

Notification System — Potential future features could be notifications about delayed projects, budget limits, or unresolved issues.

Data Visualization Improvements — Financial dashboards could be made more informative by the addition of charts and graphical reports showing budget trends and utilization.

Map Improvements — GIS map functions may be enhanced to cope with overlapping project markers and in the same time improving monitoring usability.

Project Status Improvements — Project lifecycle computation and status monitoring may be improved to give a more accurate reflection of the actual project conditions.

c. Maintenance Schedule

Frequency	Task	Responsible
Daily (Mon–Thu)	Monitor and confirm successful completion of database backups	Database Administrator
Weekly	Check API and frontend availability and monitor response time	Web Administrator
Weekly	Review security logs and monitor unauthorized access attempts	Security Administrator
Monthly	Review and update frontend and backend dependencies	Developer
Monthly	Review audit logs for unusual activities and invalid user actions	MPDO Admin
Monthly	Verify active user accounts and assigned role permissions	Security Administrator / MPDO Admin

Quarterly	Perform database maintenance and index assessment	Database Administrator
Quarterly	Review DTS integration, API connectivity, and field mappings	Web Administrator / Developer
Quarterly	Update environment variables and change JWT secret keys	Security Administrator
As Needed	Perform Alembic migration for database schema changes	Developer / Database Administrator
As Needed	Perform manual backup before major updates or deployments	Database Administrator
As Needed	Perform rollback and redeployment procedures during deployment failures	Web Administrator

d. Tools and Resources

Tool / Platform	Purpose
GitHub Repository	Source code management
GitHub Actions	Automated database backup and CI/CD workflow
Supabase	PostgreSQL database hosting and management
Render	Backend API hosting
Netlify	Frontend hosting
Alembic	Database migration management
Docker Compose	Local development and testing
pg_dump / pg_restore	Database backup and restoration
Swagger UI	API testing and documentation
Supabase Dashboard	Database inspection and monitoring

e. Roles and Responsibilities

Alubijid IT Office

The Alubijid IT Office is the main technical support group in charge of the infrastructure, security, database, and general technical operation of the PRIME System. The IT Office is split into the following administrative roles:

Database Administrator

The Database Administrator manages and maintains the system database hosted on Supabase. The tasks are:

- Checking database status and connection
- Testing database backups and conducting recovery operations
- Running database schema alterations and Alembic migrations
- Doing database fine-tuning and index checks
- Executing approved data corrections via direct database access
- Database monitoring and issue resolution
- Verifying database backup files storage and recoverability

Security Administrator

The Security Administrator works on protecting the PRIME System and maintaining its integrity. Tasks are:

- Handling JWT secret keys and securing environment variables
- Keeping track of unauthorized access and suspicious activities
- Checking audit logs and security-related reports periodically
- Implementing role-based access control policies
- Password reset policy management and maintaining account security standards
- Keeping track of dependency vulnerabilities and security patches
- Ensures sensitive credentials are properly stored and not exposed publicly

Web Administrator

The Web Administrator oversees the release, uptime, and technical operation of the web application. Duties are:

- Checking frontend and backend hosting services

- Netlify and Render deployment management
- API availability and response time monitoring
- Deployment failure and rollback handling
- Environment variables and deployment settings updating
- Integration services monitoring such as the DTS API

Developer (Technical Maintainer)

The developer takes care of the system's technical components and features by making changes and improvements. Tasks are:

- Add features and enhance functionalities
- Resolve bugs and other issues
- Carry out frontend and backend updates
- Update third-party libraries and dependencies
- Help IT Office when there is a major technical maintenance
- Use feedback for better system performance and usability
- Keep integration compatibility with external systems

f. Risk Assessment and Mitigation

The following table outlines the proactive measures designed to eliminate or reduce the impact of these risks:

Risk Identified	Possible Impact	Mitigation Strategy
Backup Failure	Having permanent loss of municipal project data.	**Routine Monitoring**: Start a weekly checklist for the Database Administrator to verify backup artifacts of GitHub Actions.
Database Migration Errors	Potentially corrupted database schema or system shutdowns.	**Staging Protocol**: Require that all Alembic migrations be run and checked in a virtual Docker environment before going live.
DTS Integration Failure	Documenting broken links and errors in reports.	**API Agility**: Keep environment variables updated and conduct quarterly API connectivity and field mappings reviews.
Service Downtime	Users are without access to their project monitoring tools.	**Health Checks**: Automated uptime monitoring should be in place and Render/Netlify status pages should be regularly checked for scheduled maintenance.

Unpatched Dependencies	Security breaches and problems with the integrity of data.	**Security Audits**: Run monthly npm audit and Python dependency checks to find and install critical security patches.
Unauthorized Access	Compromised data and loss of accountability.	**Strict Access Control:** Deploy Role-Based Access Control (RBAC) and do monthly audits of the system audit logs to keep track of unusual activities.
Data Entry Mistakes	Financial and project records are incorrect.	**Validation & Correction:** Set up a formal, documented process for Admin-led databases corrections.

By adhering to this maintenance plan, the MPDO ensures that the **PRIME System** remains a "single source of truth" that is secure, accurate, and always available for the delivery of public services.

g. Appendices

i. Checklists

Weekly

- ☐ Confirm GitHub Actions backup workflow ran successfully (Mon–Thu)
- ☐ Check Render backend service status and response times
- ☐ Check Netlify frontend deployment status
- ☐ Verify DTS document links are resolving correctly on a sample project
- ☐ Review any new entries in the system audit log

Monthly

- ☐ Run npm audit on the frontend and resolve critical vulnerabilities
- ☐ Review Python requirements.txt for outdated or vulnerable packages
- ☐ Review Supabase database connection pool metrics
- ☐ Confirm all active user accounts are still valid and appropriately assigned
- ☐ Review and archive audit logs for the previous month

ii. Log files

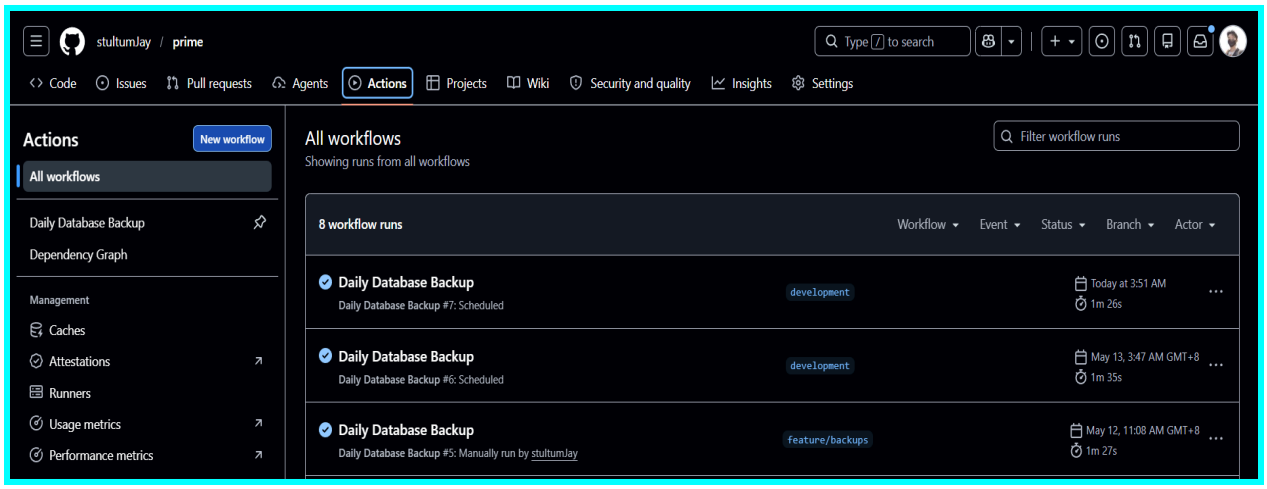


Figure 10: Automated GitHub Actions database backup workflow execution

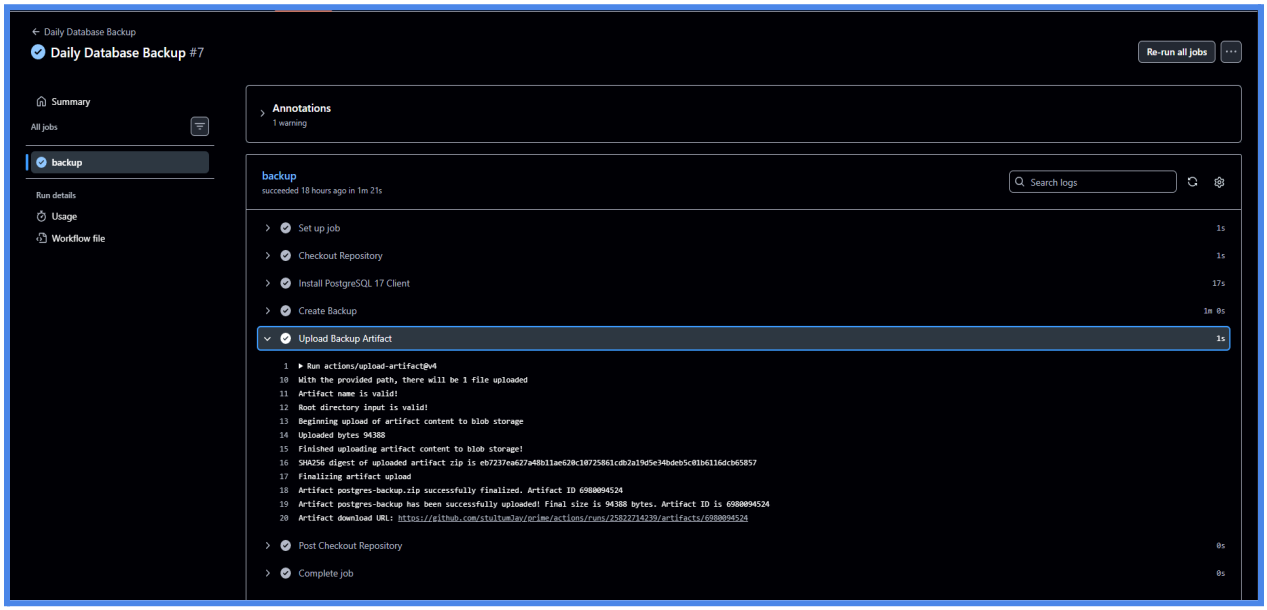


Figure 11: Generated backup artifact storage from the automated backup workflow

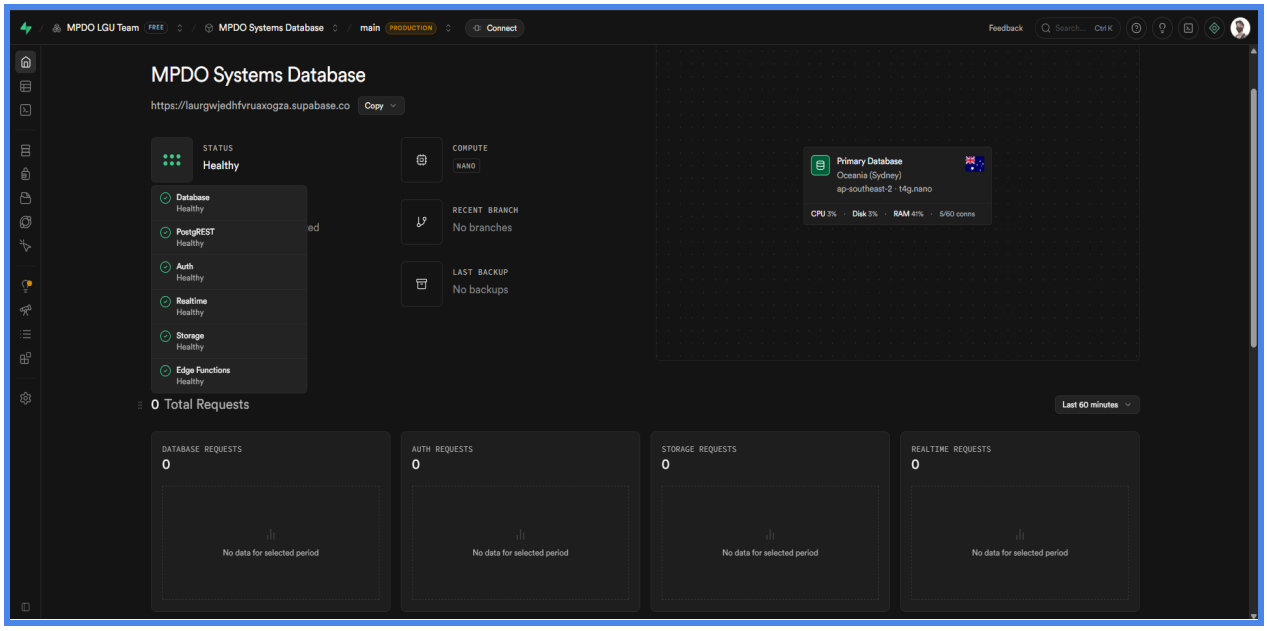


Figure 12: Supabase dashboard used for database health and monitoring checks

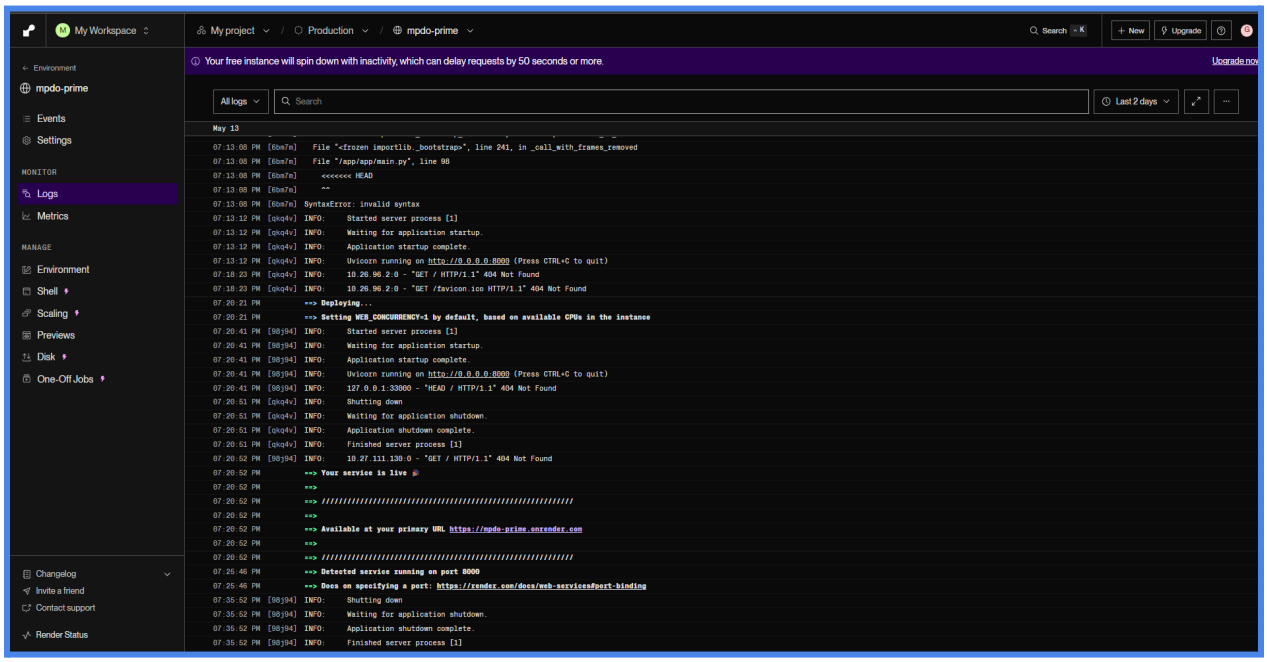


Figure 13: Render Web Service API Logs for request monitoring